Provably Lossless Acceleration of DNN Mutation Testing *via* Memoization

ALI GHANBARI, Auburn University, USA
BEN GREENMAN, University of Utah, USA
SASAN TAVAKKOL, Google Research, USA
SHIBBIR AHMED, Texas State University, USA

Mutation analysis has recently reemerged in the context of deep neural networks (DNNs) as a promising, but notoriously costly, approach for assessing test dataset adequacy. Existing techniques speed up DNN mutation testing through *lossy* approximations that trade efficiency for mutation score accuracy. This paper introduces Mure, the first provably lossless framework for accelerating DNN mutation testing *via* memoization. Mure is based on the idea that DNN mutants and the original model share substantial redundant computation, so during mutation testing, it executes only the mutated suffixes of each mutant and reuses the common prefix from the original model, which is computed only once. We give a formal account of memoized mutation testing, and prove that Mure is sound, *i.e.*, it produces results equivalent to exhaustive vanilla mutation testing, and identify basic conditions under which speed-up is guaranteed. We have implemented Mure and evaluated it on 15 DNN models of various architectures, complexities, and sizes ranging from a few thousands to millions of parameters. This provides empirical evidence that Mure reduces the computational cost of mutation testing by 44.54%, on average. We also observed that while state-of-the-art techniques tend to yield higher acceleration (up to 88.97%, on average), they come at the cost of some error in mutation score. We further analyze the effect of mutation generation selection ratio on the effectiveness of Mure and observed predictable reductions in memoization opportunities with increasing the percentage of mutated neurons. We observed that Mure offers more than 20% speed-up even when as high as 5% of the neurons are mutated.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Machine learning*.

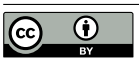
Additional Key Words and Phrases: Deep Neural Network, Mutation Analysis, Memoization, Acceleration

ACM Reference Format:
Ali Ghanbari, Ben Greenman, Sasan Tavakkol, and Shibbir Ahmed. 2026. Provably Lossless Acceleration of DNN Mutation Testing *via* Memoization. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA161 (October 2026), 22 pages. https://doi.org/10.1145/3832252

1 Introduction

Deep neural networks [28] (DNNs), have become important enabling technology in many modern software systems across many domains [2, 11, 14, 15]. Since DNNs are increasingly being used in safety- and mission-critical applications, from autonomous driving [12] to healthcare [5], their quality assurance is critical. Among various quality assurance approaches for DNNs [20, 32], testing is the most widely adopted one [57], where test data are either manually curated or automatically generated to satisfy specific adequacy requirements. However, strong performance on a given test

Authors' Contact Information: Ali Ghanbari, Auburn University, Department of Computer Science and Software Engineering, Auburn, USA, ghanbari@auburn.edu; Ben Greenman, University of Utah, Kahlert School of Computing, Salt Lake City, USA, blg@cs.utah.edu; Sasan Tavakkol, Google Research, Google Research, Irvine, USA, tavakkol@google.com; Shibbir Ahmed, Texas State University, Department of Computer Science, San Marcos, USA, shibbir@txstate.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/10-ARTISSTA161
https://doi.org/10.1145/3832252

dataset does not necessarily guarantee robustness or generalization of the models and a principled means of evaluating the quality of test data is required.

Recently, *mutation analysis* [3, 13] has been revisited in the context of DNNs [34, 37, 46, 49] as a promising technique for assessing test data quality (see §2.2 for further background). However, performing mutation analysis on DNNs is computationally expensive [9, 19, 35, 37, 45]. The potential benefits of DNN mutation analysis has motivated researchers in recent years to investigate techniques for accelerating mutation analysis [6, 7, 9, 10, 31, 33, 35, 45]. Unfortunately, all of the existing techniques, while effective, accelerate mutation analysis at the cost of incurring some error on the mutation score which can lead to errors in judging test dataset quality. Such techniques are traditionally known as *lossy* [23, 51]. In conventional software systems, there exists a rich body of research on accelerating mutation testing [43, 50], including both lossy and *lossless* techniques, *i.e.*, mutation analysis acceleration techniques that do not incur any mutation score error, which we also refer to as *sound*. However, since mutation testing for DNNs is relatively recent, lossless mutation analysis acceleration has not yet received much attention.

In this paper, we propose Mure, a provably lossless acceleration framework for DNN mutation testing. The key insight behind Mure is that mutants and the original model share portions of their computation; rather than redundantly executing those shared portions for each mutant, Mure evaluates common computation once and *safely* reuses it. Roughly speaking, given a DNN and its mutants, Mure first constructs a *layer dependence graph* to capture how layers depend on each other. Layer dependence graph is used to identify *memoization boundaries* for the mutants, *i.e.*, the collection of indices of the layers before the first mutated layer on which the layers on or after the first mutated layer depend. Mure then groups mutants by their *memoization boundaries*. This way, we essentially group the mutants based on the deepest shared prefix of the mutants whose activations can be safely reused from the original model. A *memo table* is also constructed by instrumenting the original model once per each group guided by their shared memoization boundary. Grouping mutants and constructing memo table once per group avoids creating large memo tables. Mutants in each group are then *chopped* right after their shared memoization boundary by replacing the shared prefix with a *fabricated input layer*, which feeds inputs to the layers on or after the first mutated layer. During mutation testing, the input for each chopped mutant is fetched from the memo table. This way, only the mutated suffix of the mutants will be executed. The outputs of the mutants are then used for calculating mutation score.

We formalize this process and prove that Mure is *sound*, *i.e.* lossless. In other words, executing mutants through Mure is equivalent to executing the mutants exhaustively without memoization, aka, *vanilla* mutation testing. This implies that Mure produces mutation testing outcomes that are equivalent to vanilla approach, while avoiding redundant computations. We prove a speed-up theorem showing that Mure is guaranteed to reduce execution cost whenever multiple mutants share a non-trivial prefix (see §4 for details).

We have implemented Mure in Python using Keras and Tensorflow frameworks. It is applicable to supervised classifier Keras models with a wide range of architectures. While our formal guarantees establish soundness and theoretical performance potential, they do not fully determine the practical effectiveness of Mure. Specifically, some important factors are naturally empirical, *e.g.*, the cost of model instrumentation, the overhead of memo table and mutant chopping, and external factors such as system load and stochasticity of mutation generation. Moreover, to compare Mure to prior work, empirical comparison is necessary. Thus, we conduct an extensive empirical evaluation using 15 DNN models with different architectures and complexities from different domains, including fully-connected neural networks (FCNNs), convolutional neural networks (CNNs) with and without residual blocks, and recurrent neural networks (RNNs). Our experiments investigate two research questions. First, we evaluate whether Mure delivers meaningful acceleration in practice and how it

compares to state-of-the-art approaches. We observed that, in our dataset, Mure reduces the cost of DNN mutation testing by 44.54%, on average, all the while *incurring no mutation score error*, as it was already established by our soundness theorem. We further observed that other state-of-the-art techniques, DM# [9] and BSS [45], while provide higher acceleration (63.59% and 88.97%, on average, respectively), they incur mutation score error. Second, we examine how mutation generation selection ratio with different values (*i.e.*, mutating different percentage of neurons from the original model) affects memoization potential in Mure. We observed a predictable decreasing trend: as the mutation generation selection ratio increases, the achievable speed-up gradually decreases due to reduced shared computation, but even in mutation generation selection ratios as high as 5%, Mure still offers more than 20% speed gain (see §5 for more details).

In summary, this paper makes the following contributions.

- **Technique:** We propose Mure, the first memoization-based technique for accelerating DNN mutation testing. We have implemented Mure [8] to be applicable to a wide range of Keras models. Mure is publicly available in [8].
- **Theoretical Results:** We formally define the execution semantics of memoized mutation testing and prove that Mure is lossless and equivalent to vanilla mutation testing. We also establish a speed-up theorem that shows Mure is guaranteed to yield a strict reduction in computation cost whenever redundant prefix computation exists across mutants.
- **Empirical Results:** Using 15 DNN models of varying sizes, architectures, and complexities, we evaluate the practicality of Mure in terms of efficiency and compare it to state-of-the-art mutation acceleration approaches. Additionally, we study the impact of mutation generation selection ratio on the effectiveness of Mure.

2 Background

2.1 Deep Neural Networks

Deep neural networks (DNNs) use multiple interconnected layers of transformation functions to convert inputs to outputs, wherein each layer learns successively higher level of abstraction from the training data. Each layer in a DNN consists of *neurons*, which are essentially non-linear computational units that apply an *activation function*, *e.g.*, rectified linear unit (ReLU), to the weighted sum of their inputs. The weighted edges connecting neurons are often called *synapses*.

When every neuron in a layer connects to all neurons in the next layer, we refer to the network as a fully connected neural network (FCNN). Besides FCNNs, other well-known DNN architectures include convolutional neural networks (CNNs) and recurrent neural networks (RNNs), both of which can also be represented as graphs of neurons and synapses. For instance, a convolutional layer with a $8 \times 2 \times 2 \times 4$ filter can be interpreted as 4 neurons, each summing over 32 synapses. Thus, throughout this paper we regard DNNs as graphs, allowing us to use the terms "nodes" and "neurons," as well as "edges" and "synapses," interchangeably.

2.2 DNN Mutation Analysis

Mutation analysis [3, 13] is a program analysis technique for evaluating test suite quality in conventional programs. It relies on *mutation operators*, aka *mutators*, that systematically transform program elements to generate program variants called *mutants*. These mutants are tested using a test suite to compare their output to that of the original program. If outputs differ, the mutant is marked *killed*, otherwise it is regarded as *survives*. Some mutants survive because they are semantically equivalent to the original program, hence the name *equivalent mutants*. The effectiveness, *i.e.*, mutation adequacy, of a test suite is measured by the *mutation score*, which is defined as the ratio of killed to all non-equivalent mutants. The higher the mutation score the stronger the test suite

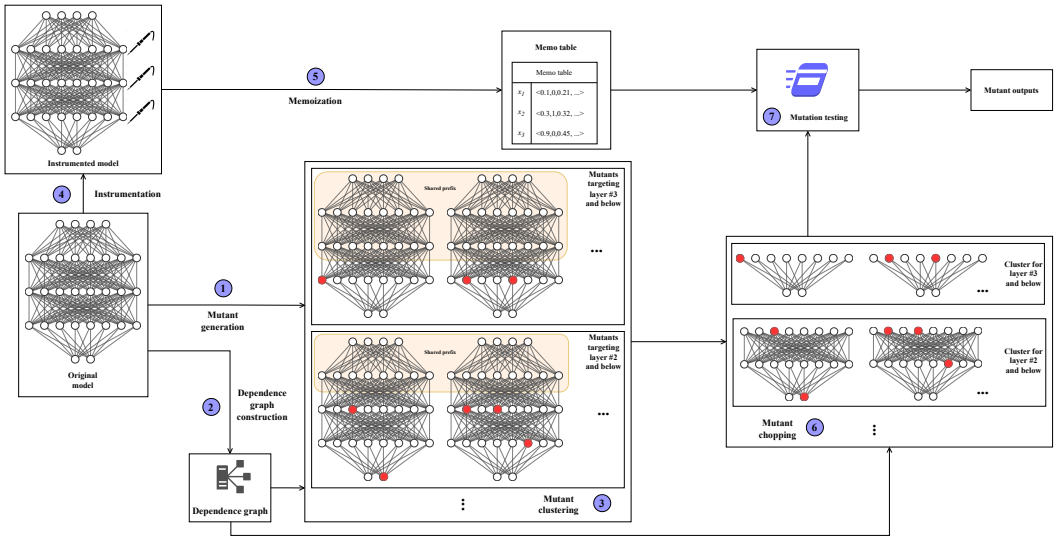


Fig. 1. Mure workflow: artifacts are represented with boxes and arrows represent control flow

is. Beyond test suite assessment, mutation analysis has been widely applied in various software engineering tasks [23, 41].

More recently, mutation analysis has been adapted for DNNs [37, 46]. Like neuron coverage [42] and its variants [36], and unlike surprise adequacy [24], DNN mutation analysis is a structural test dataset adequacy assessment method. Like its conventional counterpart, DNN mutation analysis has had plenty of applications beyond its original use in test quality assessment. These applications include adversarial sample detection and generation [17, 52], robustness evaluation [19], test data prioritization [54], model accuracy estimation [18], fault localization and repair [10, 48, 55] and modularity [7]. Specialized mutation analysis frameworks further target particular DNN domains such as autonomous driving [21] and reinforcement learning [34].

DNN mutation analysis, whether *source-based* (mutating the source code defining the DNN model) or *model-based* (mutating the neurons or their learned weights directly), is extremely costly. Given the potentials of DNN mutation analysis, several research projects in the past few years have aimed to reduce its costs (see §7 for more details). Following a taxonomy, common in the literature for mutation analysis acceleration of conventional programs [23, 51], we can classify the DNN mutation analysis acceleration techniques into two general categories: (1) lossy techniques that accelerate mutation analysis at the cost of loss in mutation score; (2) lossless, aka sound, techniques that accelerate mutation analysis without impacting mutation score. Virtually all the existing DNN mutation analysis acceleration techniques are lossy. In this paper, we introduce a novel lossless mutation analysis technique based on memoization.

3 Mure Approach

Mure is based on the idea that since multiple mutants as well as the original model share common portions, during testing we can evaluate those common portions once and reuse the results. Specifically, if two mutants of a model involve mutating the layer(s) on or after layer indexed n, the layers $0, \ldots, n-1$ are all unchanged are identical to the ones in the original model. So, to test the two mutants, instead of testing them individually and reevaluating layers $0, \ldots, n-1$ twice, we can evaluate the original model up to layer $n-1$ and then evaluate the mutants after layer $n-1$.

Algorithm 1: Mure memoized mutation testing

Input: Model M, test dataset X, a mapping $\mathcal{M}$ from mutant ids to mutants
Output: A mapping from mutant ids to their output on X

1 **Method:**
2 $dg \leftarrow$ `getDepGraph`(M)
3 $clusters \leftarrow \{\}$
4 **for** $(id, M') \in \mathcal{M}$ **do**
5 $mmi \leftarrow \min(\mu(M'))$
6 $K \leftarrow \{j \mid j \in dg[li] \wedge j < mmi \wedge li \in mmi \ldots |M.\texttt{layers}| - 1\}$
7 **if** $K \notin keys(clusters)$ **then**
8 $clusters[K] \leftarrow []$
9 $clusters[K]$.`append`(id)
10 $res \leftarrow \{\}$
11 **for** $(K, ids) \in clusters$ **do**
12 $M_i \leftarrow$ `instrument`(M, K)
13 $MT \leftarrow M_i$.`predict`(X)
14 **for** $id \in ids$ **do**
15 $M' \leftarrow \mathcal{M}[id]$
16 $mmi \leftarrow \min(\mu(M'))$
17 $M'_{\text{chopped}} \leftarrow$ `chopMutant`(M', mmi, dg, K, MT)
18 $res[id] \leftarrow M'_{\text{chopped}}$.`predict`$([MT[k] \mid k \in K])$
19 **return** res

Algorithm 2: Dependence graph calculation: procedure `getDepGraph` used in Algorithm 1

Input: Model M
Output: A dictionary dg mapping each layer index to a set of layer indices from which it receives inputs

1 **Method:**
2 **if** M *is* `Sequential` **then**
3 **return** $\{i \mapsto \{i-1\} \mid 0 < i < |M.\texttt{layers}|\} \cup \{0 \mapsto \emptyset\}$
4 $dg \leftarrow \{\}$
5 **for** i *in* $0 \ldots |M.\texttt{layers}| - 1$ **do**
6 $L \leftarrow \emptyset$
7 $l \leftarrow M.\texttt{layers}[i]$
8 **for** n *in* $l._inbound_nodes$ **do**
9 $L \leftarrow L \cup \{\texttt{layerIndexOf}(n)\}$
10 $dg[i] \leftarrow L$
11 **return** dg

Later in §4 we will demonstrate that this is expected to result in a speed up without impacting the mutation score, and in §5 we evaluate Mure empirically.

Fig. 1 illustrates Mure's workflow at a high level. Algorithm 1 is a more formal description. Mure takes a model M, a test dataset X, and a mapping $\mathcal{M}$ of mutants of M. The mapping $\mathcal{M}$ maps unique mutant identifiers to mutant objects. Mutant generation, marked with ① in Fig. 1, is explained in more details in §3.1. For each mutant M', the function μ returns the set of zero-based layer indices that have undergone mutation in M'.

Mure a constructs dependence graph for the model M by calling `getDepGraph` (Line 2 in Algorithm 1 and step ② in Fig. 1). A dependence graph is a dictionary, mapping layer index li to the set of layer indices $\{li_0, \ldots, li_n\}$ if the layer at li receives its inputs from layers indexed $li_0, \ldots, li_n$, hence depends on them. We present the details of this function in §3.2. Mure then clusters the mutants guided by the index of the earliest layer mutated in each mutant (step ③ in Fig. 1). This process is formalized in Lines 3–9 in Algorithm 1: for each mutant M' with mutant id id, we find the layer indices that come before the earliest layer mutated by M' such that some layer on or after the earliest mutated layer depends on them. The activation values of these layers, indexed by set K, called *memoization boundary*, will constitute the inputs to the *chopped mutants*, described shortly. Clusters are constructed by putting the mutant ids with equal K in the same bucket.

After grouping the mutants based on their K sets, Mure iterates through the clusters as follows. It first instruments the original model M at the layers indexed by K (step ④ in Fig. 1). The instrumented model is then applied on the given test dataset X to construct a memo table (⑤ in Fig. 1). The memo table is essentially a dictionary mapping each data point in X to the activation vectors of the layers indexed by K at that data point. Instrumentation is done by passing M to the function `instrument`, which reads outputs of the layers indexed by K. This process is explained in detail in §3.3. After constructing the memo table for a group of mutants, Mure *chops* the mutants within the cluster by discarding unmutated layers that come before the earliest mutated layer and

replacing them with a single fabricated input layer (⑥ in Fig. 1). Mure does this while respecting dependencies between the layers. The function `chopMutant`, explained in §3.4, is responsible for mutant chopping. Each chopped mutant is then tested by retrieving their inputs from the memo table (⑦ in Fig. 1). The outputs of the chopped mutants are stored in a dictionary indexed by the mutant ids. These steps are formalized in Lines 10–19 of Algorithm 1. In this paper, we use mutant outputs to calculate mutation score *à la* Ma *et al.* [37], but the results could be used for virtually any other task that relies on DNN mutation, *e.g.*, [7, 10, 52, 54].

3.1 Mutant Generation

Mure takes a set of pre-generated mutants as input. We use the mutation generation engine of DeepMutation [37] to generate the mutants, but any other mutation generator would also work. Specifically, we use DeepMutation's *Gaussian Fuzzing*, *Neuron Activation Inverse*, *Weight Shuffle*, and *Neuron Effect Block* mutators to create a set of mutants for testing. While DeepMutation offers more mutators, *e.g.*, Layer Removal [37], previous studies [22, 52] suggest that those are less effective in the sense that they are likely to create trivial mutants. Thus, we use the aforementioned set of mutators only, some of which have also been used in other works [7, 52]. These mutators may mutate more than one neuron at a time depending on the user configuration. The degree of higher mutation impact in DeepMutation is controlled by a parameter called *mutation generation selection ratio*, which specifies what percentage of neurons in a model should be selected for mutation. The default value for this parameter is 1%, *i.e.*, 1% of the neurons from the models are mutated. We study the impact of mutation generation selection ratio on the effectiveness of Mure in §5.

3.2 Constructing a Dependence Graph

Removing layers before the earliest mutation point is not always safe, especially in functional models where any layer can depend on any other layer and not necessarily the layer immediately before it. Mure addresses this challenge by identifying dependencies between the layers so that if a layer on or after the earliest mutation point depends on layer(s) before the earliest mutation point, those layers are instrumented in the original model and their outputs are fed to the chopped models during memoized mutation testing. Layer dependencies in Mure are represented as a *dependence graph*. A dependence graph is a mapping from layer indices to the set of layer indices on which the layer depends. The procedure `getDepGraph`, which is presented in Algorithm 2, is responsible for creating dependence graph. This algorithm receives a Keras model as input and returns the dependence graph for the model. In line 2, the algorithm checks if the model is a `Sequential` Keras model, in which case, the dependence graph simply maps each layer index to the layer index before it, and the 0th layer is mapped to empty set, as the input layer does not depend on other layers (Line 3). Otherwise, for models with more general architecture, *e.g.*, a model with residual blocks, the algorithm constructs the dependence graph as follows. It populates an empty mapping, by iterating through the layers and collecting the layer indices for the inbound nodes for each layer. Keras provides an attribute `_inbound_nodes` for the layer objects that contains a list of nodes, *i.e.*, neurons, whose outputs are fed into the given layer object. In lines 8–9, the algorithm iterates through the inbound neurons and adds their layer indices to a set. The function `layerIndexOf` returns the index of the layer containing a given neuron. By adding the layer indices in a set, we remove duplicate layer indices in cases where a layer depends on multiple neurons from another layer. After that, the algorithm maps the layer index to the set that it has just constructed. The mapping is returned at Line 11.

Algorithm 3: Model instrumentation: `instrument` procedure used in Algorithm 1

Input: Model M and a list of layer indices p serving as probe points
Output: An instrumented models whose outputs are the activations at layers indexed by p

```
1  Method:
2      if M is Sequential then
3          inp ← Input(shape = M.input_shape)
4          if p = [] then
5              return
                 Model(inputs = inp, outputs = inp)
6          x ← inp
7          for i ∈ 0 . . . |M.layers| − 1 do
8              x ← M.layers[i](x)
9              if i ∈ p then
10                 return Model(inputs =
                     inp, outputs = x)
11         error "Probe point not found"
12     outs ← []
13     for i ∈ 0 . . . |M.layers| − 1 do
14         l ← M.layers[i]
15         if i ∈ p then
16             outs.append(l.output)
17     return
         Model(inputs = M.inputs, outputs = outs)
```

Algorithm 4: Mutant chopping: procedure `chopMutant` used in Algorithm 1

Input: Mutant M', minimum mutated layer index mmi, layer dependence graph dg, boundary layer indices K, and memo-table MT
Output: M' chopped at layer indexed mmi taking inputs at that layer

```
1  Method:
2      if M′ is Sequential then
3          inp ← Input(shape = shape(MT[k])),
             where K = {k}
4          x ← inp
5          for j ∈ mmi . . . |M.layers| − 1 do
6              x ← M′.layer[j](x)
7          return Model(inputs = inp, outputs = x)
8      mutInp ← {k ↦ Input(shape =
         shape(MT[k])) | k ∈ K}
9      past ← {}
10     for j ∈ mmi . . . |M.layers| − 1 do
11         l ← M′.layers[j]
12         layerInp ← []
13         for p ∈ dg[j] do
14             if p < mmi then
15                 layerInp.append(mutInp[p])
16             else
17                 layerInp.append(past[p])
18         x ← l(inputs = layerInp)
19         past[j] ← x
20     return Model(inputs = [il | (_, il) ∈
         mutInp], outputs = x)
```

3.3 Model Instrumentation and Memo Table Construction

Mure records the output values, aka activations, of any layer before earliest mutation point that has dependent(s) on or after the earliest mutation point. It does so by defining a new model, called *instrumented model*, based on the existing `Sequential` or functional model. The instrumented model receives the same inputs that the original model, but it outputs the activations of a selected list of layers that we call *probe points*. The procedure `instrument`, presented in Algorithm 3, creates an instrumented model given a Keras model M and a list p of probe points. Algorithm 1 invokes this procedure by passing a memoization boundary K as probe points. If M is `Sequential`, the algorithm creates an input layer that receives the same input shape as M. If p is empty, it means that the layer right after the input layer has been mutated, so the instrumented model acts as an identity function returning its inputs (Lines 4–5). Otherwise, there is one probe point and the algorithm "stitches" layers together, *via* layer invocation, to produce the output of the instrumented model (Lines 6–10). Line 11 is a safeguard against passing an invalid p, *e.g.*, the one that does not contain a valid layer index. Handling functional models is easier in that there is no need for instantiation and stitching of the layers: the algorithm simply uses the collective outputs of the probe points as the output of the instrumented model (Lines 12–17).

The outputs of instrumented models are used to construct a memo table shared among all mutants with a common memoization boundary. More concretely, given the test dataset T and instrumented model M_i, a memo table is constructed by building the mapping $\{t \mapsto M_i(t) \mid t \in T\}$, where $M_i(t)$ returns the activations of M_i at its probe points.

3.4 Model Chopping

Model chopping is the last preparation step before Mure executes memoized mutation testing over a cluster of mutants. Given a mutant M', the index *mmi* of its earliest mutated layer, the dependence graph dg, the set of boundary layers K, and the memo table MT, the procedure `chopMutant`, described in Algorithm 4, constructs a new Keras model called the *chopped mutant*. Chopped mutant behaves like M' on the portion of the network on and after layer index *mmi*, but it no longer contains any of the unmutated prefix layers. Instead, it takes as input the activation values at the boundary layers in K, which are supplied at test time from the memo table (see §3.3). Intuitively, `chopMutant` "re-roots" the mutant at the earliest mutation point while preserving all data dependencies from the removed prefix through explicit inputs.

For `Sequential` models, chopping is straightforward because the dependence structure is linear: the algorithm fabricates a single input corresponding to the boundary layer immediately preceding *mmi*, and then reconnects the remaining layers in order, effectively treating the suffix starting at *mmi* as a smaller sequential network whose first input is the memoized activation right before *mmi* (Lines 2–7). In functional models, the layers in the suffix may depend on multiple predecessors, some of which may lie in the prefix to be removed. In this case, Algorithm 4 creates one input layer for each boundary layer in K (Line 8). It then rebuilds the mutant from layer *mmi* to the output layer by consulting the dependence graph as follows: if a layer's predecessor lies in the prefix, the algorithm uses the corresponding boundary input; otherwise, it uses the already reconstructed output of the predecessor (Lines 9–20). This incremental reconstruction preserves the original dataflow of the mutant while discarding all computation before *mmi*. The resulting chopped model behaves identically to the full mutant when evaluated with the memoized activations at the boundary layers, enabling Mure to avoid redundant recomputation of the shared prefix during mutation testing.

4 Theoretical Results

In this section, we give a more formal account of the properties of the Mure memoized mutation testing algorithm. Specifically, we present a proof of soundness, *i.e.*, Mure does not change the mutation outcome (in other words, Mure is a lossless acceleration technique), and a proof that Mure is expected to result in speed-up. We will first present the formalism for representing DNNs that both proofs rely on. We would like to emphasize that both of these proofs rely on the fact that none of our mutators physically delete any of the layers or neurons in the model nor do they add new layers or neurons. This is true for the mutators used in this paper (see §3.1). We expect that this assumption will not diminish the usefulness of our theoretical results for most real-world situations because structure-altering mutators tend to generate trivial, easily-killable, mutants [22], and are avoided by some works [7, 52].

A DNN model is a directed acyclic graph $M = (V, E, \Phi, I, O)$, where $V = \{v_0, \ldots, v_L\}$ is a set of nodes representing layers, $E \subseteq V \times V$ is a set of directed edges representing layer dependencies along which the tensors flow, $\Phi(v)$ is the layer function at v mapping tensors from the previous layer(s) to an output tensor, $I \subseteq V$ is the non-empty set of input layers, and $O \subseteq V$ is the non-empty set of output layers. In this definition, $I \neq O$. In addition, there must be a topological ordering function $\tau : V \to \mathbb{N}$ such that $(v_i, v_j) \in E$ implies $\tau(v_i) < \tau(v_j)$, for all $v_i, v_j \in V$.

A *mutant* M', created by mutating M, is defined to be $M' = (V, E, \Phi', I, O)$, wherein V, E, I, and O are unchanged, because none of the mutants alter the physical structure of the model. The layer function, however, is defined as follows.

$$\Phi'(v) = \begin{cases} \Phi(v) & ; v \in \mu(M') \\ \text{mutated layer function} & ; \text{otherwise} \end{cases}$$

In this definition, $\mu(M') \subseteq V$ is a function that returns a subset of layers that have been the subject of mutation. Based on this definition, Φ' is identical to Φ except at the mutated layers where Φ' use the layer function for the mutated layers. The layer function for the mutated layers is left *unspecified*, and none of our proofs need a precise characterization of the function. The topological ordering τ allows us to formally define *earliest mutated layer* of M' as $mmi = \arg\min_{v \in \mu(M')} \tau(v)$.

We define *memoization boundary*, *i.e.*, the required input set for a chopped mutant (defined shortly), as the set of all *unmutated* layers whose outputs are needed to compute the mutated region. This set is defined to be $K = \{p \mid (p, v) \in E \text{ such that } \tau(v) \geq \tau(mmi) \text{ and } \tau(p) < \tau(mmi)\}$.

The *activation*, *i.e.*, the output value, for each layer $v \in V$ in the original model M is defined recursively as follows.

$$\mathbf{a}_v(\mathbf{x}) = \begin{cases} \mathbf{x} & ; v \in I, \\ \Phi(v)(\mathbf{a}_p(\mathbf{x})_{p \in Pred(v)}) & ; \text{otherwise} \end{cases}$$

where $Pred(v) = \{p \mid (p, v) \in E\}$ are the immediate predecessors of v. We use the notation $M(\mathbf{x})$, defined as $(\mathbf{a}_v(\mathbf{x})_{v \in O})$, to represent the result of applying M on a data point $\mathbf{x}$. We define a memo table as function that maps each $k \in K$ to the tensor $\mathbf{a}_k(\mathbf{x})$, *i.e.*, the activation of layer k on input $\mathbf{x}$. Similar to $\mathbf{a}_v$, we define $\mathbf{b}_v(\mathbf{x})$, the activation for each layer $v \in V$ in the mutated model M' recursively using $\Phi'(v)$ in place of $\Phi(v)$ and $\mathbf{b}_p(\mathbf{x})$ instead of $\mathbf{a}_p(\mathbf{x})$. We also define $M'(\mathbf{x})$ as $(\mathbf{b}_v(\mathbf{x})_{v \in O})$ representing the result of applying the mutant M' on a data point $\mathbf{x}$.

A *chopped mutant*, $M^\star$, is a DNN whose inputs consist of one tensor per boundary node $k \in K$, each having the same shape as the activation of layer $k \in K$ in the original model. More concretely, a chopped mutant is a directed acyclic graph $M^\star = (V^\star, E^\star, \Phi^\star, I^\star, O)$, where $I^\star = \{u_k \mid k \in K\}$ where u_k is a fresh input layer, O is the set of output layer nodes as in the original model M, and $V^\star = \{v \mid v \in V \text{ and } v \text{ is reachable from } mmi \text{ in } M\} \cup I^\star \cup O$. The set $E^\star \subseteq V^\star \times V^\star$ is defined as follows. For each $v \in V^\star$, $(p, v) \in E^\star$ if and only if (1) $(p, v) \in E$ and $p \in V^\star$, or (2) $p = u_k$ for some $k \in K$ and $(k, v) \in E$. Finally, $\Phi^\star(v)$ is defined as follows.

$$\Phi^\star(v) = \begin{cases} \Phi'(v) & ; v \in \mu(M'), \\ \Phi(v) & ; v \notin \mu(M') \text{ and } v \in (V^\star \cap V), \\ \mathbf{id} & ; v \in I^\star, \end{cases}$$

where **id** is the identity function returning the tensor it receives from the input without any computation. The function $\Phi^\star$ behaves identical to Φ' for all mutated layers and it behaves identical to the original Φ for all unmutated layers. The function models the behavior of input layers as identity functions. Given this layer function, we define the *activation* for each layer $v \in V^\star$ in the chopped model recursively as follows.

$$\mathbf{b}^\star_v(\mathbf{x}) = \begin{cases} \mathbf{x} & ; v \in I^\star, \\ \Phi^\star(v)(\mathbf{b}^\star_p(\mathbf{x})_{p \in Pred^\star(v)}) & ; \text{otherwise} \end{cases}$$

where $Pred^\star(v) = (Pred(v) \cap V^\star) \cup \{u_k \mid k \in K \text{ and } k \in Pred(v)\}$. Finally, we define $M^\star(\mathbf{x})$ as $(\mathbf{b}^\star_v(\mathbf{x})_{v \in O})$ to represent the result of applying the chopped mutant $M^\star$ on a data point $\mathbf{x}$.

4.1 Soundness

Our soundness proof relies on one more assumption that for any unmutated layer v, the function $\Phi(v)$ is deterministic and depends only on its inputs. In particular, for inference in Keras, where `BatchNorm` and `Dropout` are in inference mode and no stateful RNNs are used or their states are reset, $\mathbf{a}_v(\mathbf{x})$ depends only on the inputs it receives, *not* on prior test inputs or model state. Please note that this assumption does not create any limitation for the applicability of our theoretical

results, because as long as we run our models in inference mode, and do not load models with `stateful=True` (or set it to `False` before applying Mure), this assumption will hold.

The soundness proof states that the output of Algorithm 4 is equal to that of the full mutant. This way, instead of testing full mutants, one may test the chopped mutants and get the same mutation testing results. Soundness directly follows from several lemmas described shortly: first we show that the output of Algorithm 4 is equivalent to the theoretical model of chopped mutant that we defined in §4 (Lemma 4.2), in that they both compute the same function, we then demonstrate that the theoretical model of chopped mutant is functionally equivalent to the full, un-chopped mutant (Lemma 4.3). These results together imply that the output of Algorithm 4 is functionally equivalent to the full, un-chopped mutant (Theorem 4.4), *i.e.*, they both compute the same function.

Throughout this section, Let $M = (V, E, \Phi, I, O)$ be a DNN model, wherein layers set $V = \{v_0, \ldots, v_L\}$ is ordered using Keras' topological order τ, *i.e.*, $\tau(v_i) = i$, for all $0 \leq i \leq L$. Before presenting our main results, we need to state and prove a helper lemma which concerns the correctness of Algorithm 2.

Lemma 4.1 (Dependence Graph Correctness). *Let* $dg = \texttt{getDepGraph}(M)$. *Then for every index* i, *we have* $dg[i] = \{j \mid (v_j, v_i) \in E\}$.

Proof. We proceed by a case analysis on the type of the model M: (1) M is a `Sequential` model; (2) M is a more general functional model. In case (1), the layers are linearly ordered according to τ and they are connected to each other in that order, so that all edges in E have the form (v_{i-1}, v_i). Additionally, according to Line 3 of Algorithm 2, we have $dg[i] = \{i-1\}$, for $i > 0$, and $dg[0] = \emptyset$. Thus, we can rewrite $dg[i]$ as $\{j \mid (v_j, v_i) \in E\}$, which is exactly the result that we were looking for.

In case (2), while the layers are linearly ordered according to τ, they are not necessarily connected to each other in that order. Based on lines 6–10 of Algorithm 2, $dg[i]$ contains layer indices of the inbound neurons to the i-th layer, so $dg[i] = \{\tau(p) \mid (p, v_i) \in E\}$. Since $\tau(v_j) = j$, for all $0 \leq j \leq L$, we can rewrite $dg[i]$ as $\{j \mid (v_j, v_i) \in E\}$. □

We now use this lemma to show that the output of Algorithm 4 is functionally equivalent to the theoretical model of chopped mutant.

Lemma 4.2 (Mutant Chopping Correctness). *Let* $M' = (V, E, \Phi', I, O)$ *be a mutant of* M, *and* $mmi \in \{0, \ldots, L\}$ *be the index of the earliest mutated layer in* M'. *Let* K *be the memoization boundary associated with* M' *and* mmi, MT *be the memo table associated with* M *and* K, *and* $M^\star = (V^\star, E^\star, \Phi^\star, I^\star, O^\star)$ *be the chopped mutant, as defined in §4. Let* $dg = \texttt{getDepGraph}(M)$, *and* $M^\star_{Alg} = \texttt{chopMutant}(M', mmi, dg, K, MT)$, *such that* $M^\star_{Alg} = (V^\star_{Alg}, E^\star_{Alg}, \Phi^\star_{Alg}, I^\star_{Alg}, O^\star_{Alg})$. *Then* $M^\star$ *and* $M^\star_{Alg}$ *compute the same function.*

Proof. We show that there exists a bijection $h : M^\star \rightarrow M^\star_{Alg}$, such that:

(a) *Nodes preserved*: For each boundary node $k \in K$, $h(u_k)$ is the input layer created in Algorithm 4 for k. For each internal node $v \in V \cap V^\star$, $h(v)$ is the node constructed based on v.
(b) *Edge structure preserved*: For each $p, v \in V$, $(p, v) \in E^\star \iff (h(p), h(v)) \in E^\star_{Alg}$.
(c) *Layer functions preserved*: For each $v \in V^\star$, $\Phi^\star(v) = \Phi^\star_{Alg}(h(v))$.

The proof proceeds by a case analysis on the type of mutant M': (1) the mutant M' is `Sequential` model; (2) M' is a more general functional model.

***Case (1)**: (a) Nodes preserved.* Since M' is `Sequential`, we have $K = \{v_{mmi-1}\}$, and hence $I^\star = \{u_{v_{mmi-1}}\}$. Algorithm 4 creates exactly one input node $inp = \texttt{Input}(\texttt{shape} = \texttt{shape}(MT[v_{mmi-1}]))$ at Line 3. We define $h(u_{v_{mmi-1}}) = inp$, which clearly is a bijective mapping. To prove that h maps each node at the mutated suffix of the mutant to one and only one node in the chopped mutant, we

first note that $V \cap V^\star = \{v_j \mid j \geq mmi\}$. At lines 4–6, Algorithm 4 creates the chain: $x_{mmi-1} = inp$, $x_i = v_j(x_{i-1})$, for each $mmi \leq j \leq L$. Note that the notation $v_j(\cdot)$ is borrowed from Python, which is equivalent to calling `__call__` method of the layer object v_j. This call results in reusing layer v_j in the created chopped mutant, without cloning the layer object, and wiring x_{i-1} to x_i. We now define $h(v_j) = x_j$, for $j \geq mmi$. This is also a bijective mapping.

(b) Edge structure preserved. For a `Sequential` mutant, $E^\star = \{(u_{v_{mmi-1}}, v_{mmi})\} \cup \{(v_{j-1}, v_j) \mid j > mmi\}$. Algorithm 4 constructs exactly these edges: At line 3 and the first iteration of the **for**-loop at line 5, it connects $inp = h(u_{v_{mmi-1}})$ to $h(v_{mmi})$. In subsequent iterations of the loop, it connects $h(v_j)$ to $h(v_{j+1})$. Therefore, $(p, v) \in E^\star \iff (h(p), h(v)) \in E^\star_{Alg}$.

(c) Layer functions preserved. For memoization boundary nodes, $\Phi^\star(u_{v_{mmi-1}}) = \mathbf{id}$. The node inp = `Input(shape = shape(`$MT[v_{mmi-1}]$`))` created by Algorithm 4 also implements identity. Thus, $\Phi^\star_{Alg}(inp) = \mathbf{id}$. For each internal node v_j, $\Phi^\star(v_j) = \Phi'(v_j)$. Algorithm 4 also reuses the nodes from the mutant M', through the calling and chaining mechanism described above. So, $\Phi^\star_{Alg}(h(v_j)) = \Phi'(v_j)$, for each $j \geq mmi$. Therefore, $\Phi^\star(v) = \Phi^\star_{Alg}(h(v))$, for each $v \in V^\star$.

***Case (2)**: (a) Nodes preserved.* For chopped mutant $I^\star = \{u_k \mid k \in K\}$. At line 8, Algorithm 4 constructs the mapping $mutInp[k]$ = `Input(shape = shape(`$MT[k]$`))`, for each $k \in K$. We set $h(u_k) = mutInp[k]$, for $k \in K$. These are clearly bijective mappings. To prove that h maps each node at the mutated suffix of the mutant to one and only one node in the chopped mutant, we first note that $V \cap V^\star = \{v_j \mid j \geq mmi\}$. At lines 10–19, Algorithm 4 defines $past[j] = v_j$(inputs constructed at lines 12–17) for each $j \in \{mmi, \ldots, L\}$. As we discussed in the `Sequential` case, $v_j(\cdot)$, where $v_j \in V$, reuses v_j in the construction of $M^\star_{Alg}$ without cloning the layer. We define $h(v_j) = past[j]$, which clearly is a bijective mapping.

(b) Edge structure preserved. We note that $Pred^\star(v) = (Pred(v) \cap V^\star) \cup \{u_k \mid k \in K \cap Pred(v)\}$. It follows from Lemma 4.1 that $Pred(v_j) = \{v_p \mid p \in dg[j]\}$. At lines 14–18, for each $p \in dg[j]$, where $j \in \{mmi, \ldots, L\}$, Algorithm 4 connects $mutInp[p] = h(u_{v_p})$ to $past[j] = h(v_j)$, if $p < mmi$ (hence $v_p \in K$), and it connects $past[p] = h(v_p)$ to $past[j] = h(v_j)$, if $p \geq mmi$. After the **for**-loop of line 13, the incoming edges for $past[j]$ are emanating from the nodes $\{h(v_p) \mid p \in dg[j] \text{ and } p \geq mmi\} \cup \{h(u_{v_k}) \mid k \in dg[j] \text{ and } k < mmi\}$. Therefore, we have $(p, v_j) \in E^\star \iff (h(p), h(v_j)) \in E^\star_{Alg}$

(c) Layer functions preserved. This proof proceeds by a straightforward generalization of the sequential case to compare boundary nodes u_k and the nodes $mutInp[k]$.

□

We next show that the theoretical chopped mutant is equivalent to the full, un-chopped mutant.

Lemma 4.3 (Full and Chopped Mutant Equivalence). *Let $M' = (V, E, \Phi', I, O)$ be a mutant of M, and $mmi \in \{0, \ldots, L\}$ be the index of the earliest mutated layer in M'. Let K be the memoization boundary associated with M' and mmi, MT be the memo table associated with M and K, and $M^\star = (V^\star, E^\star, \Phi^\star, I^\star, O^\star)$ be the chopped mutant, as defined in §4. We have $M^\star(\mathbf{a}_k(\mathbf{x})_{k \in K}) = M'(\mathbf{x})$, for every data point $\mathbf{x}$.*

Proof. By expanding the definitions of model applications, we can see that the proof goal is $(\mathbf{b}^\star_v(\mathbf{a}_k(\mathbf{x})_{k \in K})_{v \in O}) = (\mathbf{b}_v(\mathbf{x})_{v \in O})$, for every data point $\mathbf{x}$. We prove this via a stronger proposition that considers all layers (including the output layer): for every $t \in \{mmi, mmi+1, \ldots, L\}$, and every data point $\mathbf{x}$, we have $\mathbf{b}^\star_{v_t}(\mathbf{a}_k(\mathbf{x})_{k \in K}) = \mathbf{b}_{v_t}(\mathbf{x})$. Note that $\{v_t \mid mmi \leq t \leq L\} = V^\star \cap V$. We proceed by induction on the topological ordering τ over $V^\star \cap V$.

Induction Basis. Let $t = mmi$. In this case, we want to prove $\mathbf{b}^{\star}_{v_{mmi}}(\mathbf{a}_k(\mathbf{x})_{k\in K}) = \mathbf{b}_{v_{mmi}}(\mathbf{x})$. By expanding the $\mathbf{b}^{\star}_{v_{mmi}}$ and $\mathbf{b}_{v_{mmi}}$, we get $\Phi^{\star}(v_{mmi})(b^{\star}_p(\mathbf{a}_k(\mathbf{x})_{k\in K})_{p\in Pred^{\star}(v_{mmi})}) = \Phi'(v_{mmi})(\mathbf{b}_p(\mathbf{x})_{p\in Pred(v_{mmi})})$. Since v_{mmi} is, by definition, mutated, *i.e.*, $v_{mmi} \in \mu(M')$, $\Phi^{\star}(v_{mmi})$ evaluates to $\Phi'(v_{mmi})$. Additionally, since $Pred^{\star}(v_{mmi}) \subseteq I^{\star}$, *i.e.*, all the predecessors of the earliest mutated layer are the inputs to the chopped mutant, by definition, $b^{\star}_p$ is the identity function for each $p \in Pred^{\star}(v_{mmi})$. Furthermore, since nodes in $Pred(v_{mmi})$ are not mutated, we have $\mathbf{b}_p(\mathbf{x})_{p\in Pred(v_{mmi})} = \mathbf{a}_p(\mathbf{x})_{p\in Pred(v_{mmi})}$. This is because the full mutant is equal to the original model before the earliest point of mutation. Thus, we get equal expressions on the two sides of equation: $\Phi'(v_{mmi})(\mathbf{a}_p(\mathbf{x})_{p\in Pred(v_{mmi})}) = \Phi'(v_{mmi})(\mathbf{a}_p(\mathbf{x})_{p\in Pred(v_{mmi})})$.

Induction Step. Take an index $t > 1$, where $mmi \le t < L$. Assume that for every index $j < t$ the above proposition holds. That is to say, $\mathbf{b}^{\star}_{v_j}(\mathbf{a}_k(\mathbf{x})_{k\in K}) = \mathbf{b}_{v_j}(\mathbf{x})$, for every data point $\mathbf{x}$. This is our *induction hypothesis*.

We need to prove $\mathbf{b}^{\star}_{v_t}(\mathbf{a}_k(\mathbf{x})_{k\in K}) = \mathbf{b}_{v_t}(\mathbf{x})$ for every data point $\mathbf{x}$. We start by expanding the definitions of $\mathbf{b}^{\star}_{v_t}$ and $\mathbf{b}_{v_t}$. Given that v_t is not an input layer, we have $\Phi^{\star}(v_t)(\mathbf{b}^{\star}_p(\mathbf{a}_k(\mathbf{x})_{k\in K})_{p\in Pred^{\star}(v_t)}) = \Phi'(v_t)(\mathbf{b}_p(\mathbf{x})_{p\in Pred(v_t)})$. Now we have two cases to examine: (1) $\tau(p) \ge mmi$, meaning that the predecessor of v_t is not an input to the chopped mutant; (2) $\tau(p) < mmi$, meaning that the predecessor of v_t is outside of the chopped region, so p must be an input to the chopped mutant (the left-hand side of the equation), *i.e.*, we have $p \in I^{\star}$ for the chopped mutant and $p \in K$ for the full mutant (the right-hand side of the equation).

Case (1) directly follows from the induction hypothesis. To prove case (2), we fix $p \in K$ and consider $u_p \in I^{\star}$ as the corresponding input layer for the chopped mutant. So, what we want to prove is $\Phi^{\star}(v_t)(\mathbf{b}^{\star}_{u_p}(\mathbf{a}_p(\mathbf{x}))) = \Phi'(v_t)(\mathbf{b}_p(\mathbf{x}))$. Note that $\mathbf{a}_p$ denotes the memo table entry that is fed to u_p. Now since $\mathbf{b}^{\star}_{u_p} = \mathbf{id}$, our proof obligation reduces to $\Phi^{\star}(v_t)(\mathbf{a}_p(\mathbf{x})) = \Phi'(v_t)(\mathbf{b}_p(\mathbf{x}))$. But since $p \in K$, which is outside of the mutated region, we have $\mathbf{b}_p(\mathbf{x}) = \mathbf{a}_p(\mathbf{x})$. This is because the full mutant is equal to the original model before the earliest point of mutation. Thus, we get $\Phi^{\star}(v_t)(\mathbf{a}_p(\mathbf{x})) = \Phi'(v_t)(\mathbf{a}_p(\mathbf{x}))$. To complete the proof, we do yet another case analysis: (a) $v_t \in \mu(M')$, meaning that v_t is a mutated layer; (b) $v_t \notin \mu(M')$, meaning that v_t is not mutated. In the first case, we have $\Phi^{\star}(v_t) = \Phi'(v_t)$, by definition. In the second case, we know that $v \in (V^{\star} \cap V)$, so both $\Phi^{\star}(v_t)$ and $\Phi'(v_t)$ will evaluate to $\Phi(v_t)$. This completes the proof. □

Finally, we are in a position where we can state our main soundness result.

Theorem 4.4 (Soundness). *Let $M' = (V, E, \Phi', I, O)$ be a mutant of M, and $mmi \in \{0, \ldots, L\}$ be the index of the earliest mutated layer in M'. Let K be the memoization boundary associated with M' and mmi, MT be the memo table associated with M and K, as defined in §4. Let $dg = \texttt{getDepGraph}(M)$, and $M^{\star}_{Alg} = \texttt{chopMutant}(M', mmi, dg, K, MT)$, such that $M^{\star}_{Alg} = (V^{\star}_{Alg}, E^{\star}_{Alg}, \Phi^{\star}_{Alg}, I^{\star}_{Alg}, O^{\star}_{Alg})$. For every data point $\mathbf{x}$, we have $M^{\star}_{Alg}((\mathbf{a}_k(\mathbf{x})_{k\in K}) = M'(\mathbf{x})$.*

Proof. Directly follows from Lemmas 4.2 and 4.3. □

4.2 Acceleration

We now prove that testing chopped mutants is faster than testing full mutants. We use an abstract evaluation-cost function $cost(F)$, that represents the computational cost of evaluating DNN model F once on a given input data point $\mathbf{x}$ and that satisfies the following *cost function axioms*:

- **C1: Non-negativity:** $cost(F) \ge 0$ for every DNN model F.
- **C2: Additivity:** If a model F can be decomposed as a prefix P and a suffix S, such that running F on an input amounts to running P on the input and then feeding its output(s) to S, then $cost(F) = cost(P) + cost(S)$.

- **C3: Invariance:** If two DNN models F and G have isomorphic computation graphs, *i.e.*, they have the same layers, same connectivity, and same parameters, then $cost(F) = cost(G)$.

As presented in §3, Mure clusters mutants based on their K values, which is the set of memoization boundary nodes. We first express the acceleration results for only one, non-empty, cluster, denoted C in this section, as a lemma. Proof of acceleration of multiple mutant clusters directly follows from this result. Let $M \downarrow K$ denote the model obtained by running M only up to the boundary nodes K, which is the shared prefix needed to calculate the memo table for the mutants in C, *i.e.*, $(\mathbf{a}_k(\mathbf{x})_{k \in K})$.

Lemma 4.5 (In-Cluster Acceleration). *Given a DNN model M, a set C of mutants, a set K of memoization boundary nodes, we have $cost(M \downarrow K) + \sum_{M' \in C} cost(M^{\star}_{Alg}) \leq \sum_{M' \in C} cost(M')$, for every input data point $\mathbf{x}$. This inequality is strict if $|C| > 1$ and $cost(M \downarrow K) > 0$.*

Proof. Take an arbitrary input data point $\mathbf{x}$. Since all the mutants in C share the same earliest mutated layer index, *i.e.*, *mmi*, the following holds for the mutants in C by construction: for all $M' \in C$, all layers v with $\tau(v) < mmi$ are unmutated, and hence are identical to M. Let P denote the prefix submodel consisting of these unmutated layers up to the boundary nodes K. By definition, evaluating $M \downarrow K$ on $\mathbf{x}$ is equivalent to evaluating P on $\mathbf{x}$. Thus, we have:

$$cost(M \downarrow K) = cost(P). \tag{1}$$

For each mutant $M' \in C$, we denote the suffix of submodel, *i.e.*, everything after the memoization boundary nodes, as $S_{M'}$. Then each full mutant M' decomposes into P and $S_{M'}$. To produce the output of M' we first apply the prefix P on the input data point $\mathbf{x}$, and its activations at K are then fed to $S_{M'}$. Then by C2, we have: $cost(M') = cost(P) + cost(S_{M'})$, for all $M' \in C$. But $S_{M'}$ is the same as $M^{\star}$, and according to Lemma 4.2, $M^{\star}$ is isomorphic to $M^{\star}_{Alg}$, so by C3, we have $cost(S_{M'}) = cost(M^{\star}) = cost(M^{\star}_{Alg})$. Thus, we have: $cost(M') = cost(P) + \sum_{M' \in C} cost(M^{\star}_{Alg})$. The total cost of *vanilla mutation testing*, *i.e.*, testing with no memoization, is then $\sum_{M' \in C} cost(M')$. Using 1, we can rewrite this as: $\sum_{M' \in C} cost(M') = \sum_{M' \in C}(cost(P)+cost(M^{\star}_{Alg})) = |C| \times cost(P) + \sum_{M' \in C} cost(M^{\star}_{Alg})$ Unlike vanilla mutation testing, Mure evaluates $M \downarrow K$ once at the cost of $cost(P)$, and for each mutant $M' \in C$ it runs the chopped mutant $M^{\star}_{Alg}$. Thus, the total cost of memoized mutation testing for the cluster C is $cost(P) + \sum_{M' \in C} cost(M^{\star}_{Alg})$. By subtracting the memoized mutation-testing cost from the vanilla mutation-testing cost, we obtain $\sum_{M' \in C} cost(M') - (cost(P) + \sum_{M' \in C} cost(M^{\star}_{Alg}))$. This simplifies to $|C| \times cost(P) + \sum_{M' \in C} cost(M^{\star}_{Alg}) - (cost(P) + \sum_{M' \in C} cost(M^{\star}_{Alg}))$, and with a bit of algebra, we get $(|C| - 1) \times cost(P)$. By C1, $cost(P) \geq 0$, and therefore the difference is non-negative. Hence, $cost(P) + \sum_{M' \in C} cost(M^{\star}_{Alg}) \leq \sum_{M' \in C} cost(M')$. Finally, by Equation (1), $cost(P) = cost(M \downarrow K)$, so $cost(M \downarrow K) + \sum_{M' \in C} cost(M^{\star}_{Alg}) \leq \sum_{M' \in C} cost(M')$. Moreover, if $|C| > 1$ and $cost(M \downarrow K) > 0$, then $(|C| - 1) \cdot cost(P) = (|C| - 1) \cdot cost(M \downarrow K) > 0$. Thus, the memoized cost is strictly smaller than the vanilla cost, and the inequality above is strict. □

The above lemma concerns a single cluster of mutants that share the same *mmi* and K. We can generalize this result to multiple clusters in a straightforward manner.

Theorem 4.6 (Acceleration). *Given a DNN model M and a set of mutants $\{M^{(1)}, \ldots, M^{(n)}\}$. Assume that these mutants are clustered into p clusters $C_1, \ldots, C_p$, which their respective boundary node sets $K_1, \ldots, K_p$ and earliest mutated layer indices $mmi_1, \ldots mmi_p$. We have $\sum_{i \in \{1,\ldots,p\}} cost(M \downarrow K_i) + \sum_{M' \in C_i} cost(M^{\star}_{Alg_i}) \leq \sum_{i \in \{1,\ldots,n\}} cost(M^{(i)})$, for every input data point $\mathbf{x}$. This inequality is strict, if at least one $|C_i| > 1$ and $cost(M \downarrow K_i) > 0$.*

Proof. Directly follows from axiom C1 and Lemma 4.5. □

Theorem 4.6 proves that there is a speed up given a set of *pre-constructed* chopped mutants, but it does not determine how much acceleration should be expected. In particular, this theorem does not take into account the costs associated with (1) constructing chopped mutants; (2) forming clusters; (3) constructing memo table; and (4) Keras modeling building as well as running the models on a physical computer. Additionally, it does not tell us how the percentage of neurons mutated impacts the amount of acceleration. Therefore, conducting experiments is necessary for getting a sense of how much speed up should be expected given all the aforementioned factors.

5 Experiments

Our theoretical results in §4 provide guarantees that Mure is lossless, and that Mure is expected to result in speed up. However, many details such as the amount of speed up as well as how it compares against related work are not clear. We answer the remaining questions empirically by investigating the following research questions (RQs).

- **RQ1:** How much speed up does Mure offer compared to the baseline approaches?
- **RQ2:** How does mutation generation selection ratio impact speed gain by Mure?

In RQ1, we compare acceleration achieved by Mure to that of two state-of-the-art approaches DM# [9] and BSS [45] on 7 models. We observed that Mure consistently accelerates DNN mutation testing across all models, with an speed gain of 44.54%. While DM# and BSS provide higher acceleration (63.59% and 88.97%, on average, respectively), they incur mutation score error.

In RQ2, we provide empirical evidence that the speed-up provided by Mure decreases gradually as the mutation generation selection ratio increases. The results also indicate that while mutation generation selection ratio reduces memoization potential, it does not eliminate the practicality of Mure even in mutation generation selection ratios as high as 5%.

5.1 Baseline Approaches

We compare Mure to three baseline approaches: (1) vanilla approach, which tests mutants exhaustively; (2) DM# [9], which clusters mutants based on behavioral similarity to test representatives from each cluster and reuse their results for all the mutants in the clusters they represent; and (3) BSS [45], which tests the mutants using only a subset of data points around the decision boundary of the original model rather than the whole test dataset. These techniques are the most effective representatives of a broad range of related techniques. DM# is the latest representative of techniques that speed up mutation testing by testing fewer mutants, *e.g.*, neuron and mutant clustering [7, 35], random mutant selection [10], mutator selection [6], and higher-order mutation [31]. Furthermore, BSS represents techniques that accelerate mutation testing by reducing mutant inputs, *e.g.*, random sample selection [9].

5.2 Measures

We quantize *speed-up*, aka *acceleration* or *gain*, as $\frac{t_v - t}{t_v}$, where t_v is the vanilla mutation testing time, while t is the mutation testing time for the acceleration techniques, *e.g.*, Mure, DM#. Both t_v and t are measured in seconds. We would like to emphasize that the measured time for Mure includes all preprocessing and bookkeeping overheads, including dependence-graph construction, mutant clustering, model instrumentation, memo-table construction, and mutant chopping. Therefore, the reported gains are net gains after accounting for any overhead Mure may have.

The second measure that we use is *mutation score*. We calculate mutation score *à la* Ma *et al.* [37]: $\frac{1}{|M|\times|L|}\sum_{\mu\in M}|\text{killingLabels}(\mu)|$, where M is the set of mutants that is obtained by applying a set of mutators on a given DNN model m, and $L = \{\text{label(t)} \mid t \in T\}$ is the set of labels in a test dataset T, wherein the function 'label' returns the ground-truth label for the data points in T. For any mutant

Table 1. Models benchmark. # Train and # Test represent the number of data points in the train and test datasets, respectively, and # Cls denotes the number of classes/labels in the test dataset.

Architecture	Dataset	Scope	Size	# Train	# Test	# Cls	Test Acc.
LeNet-5	SVHN	RQ1	62006	73257	26032	10	0.8440
MobileNetV2	Caltech-101		334886	7316	1828	102	0.6461
	CIFAR-10		287690	50000	10000	10	0.7233
ResNet10	Caltech-101		5033446	7316	1828	102	0.6723
	CIFAR-10		287690	50000	10000	10	0.8132
RNN	IMDB		2642562	2642562	25000	2	0.8083
	Reuters		2648282	8982	2246	90	0.6399
FCNN	EMNIST	RQ2	45676	124800	20800	26	0.8794
	FMNIST		44860	60000	10000	10	0.8779
	KMNIST		44860	60000	10000	10	0.8698
	MNIST		44860	60000	10000	10	0.9742
LeNet-5	EMNIST		45786	124800	20800	26	0.9197
	FMNIST		44426	60000	10000	10	0.8853
	KMNIST		44426	60000	10000	10	0.9410
	MNIST		44426	60000	10000	10	0.9899

$\mu \in M$, killingLabels(μ) is defined to be $\{\text{label}(t) \mid t \in T \text{ and } \text{kill}(\mu, t)\}$. A mutant μ is said to be *killed* by a test data point $t \in T$, denoted by the predicate kill(μ, t), if $\text{argmax}(m(t)) = \text{label}(t)$ and $\text{argmax}(\mu(t)) \neq \text{label}(t)$, where $\text{argmax}(m(t))$, or $\text{argmax}(\mu(t))$, denotes the label predicted by the model m, or its mutant μ, for t.

For lossy approaches, the *mutation score error* or *loss*, *i.e.*, the percentage of deviation of accelerated mutation score from vanilla mutation score, is calculated as $\frac{|MS_V - MS|}{MS_V}$, where MS_V is the mutation score obtained using vanilla approach, while MS represents the mutation score obtained using a lossy acceleration technique. For Mure, mutation score error is zero by design, and it is *not* because DeepMutation-style mutation score is aggregate and potentially obscures minor behavioral deviations. It is solely because chopped mutants generated by Mure produce exactly the same outcome as original mutants.

5.3 Benchmark and Setup

Table 1 lists the DNN models used in the experiments, where each row represents a model. We have used six types of DNN architectures in our experiments: 4-layer FCNN, LetNet-5 [29], ResNet-10 [15], MobileNetV2 [44], and RNN with LSTM layers. We have use standard LetNet-5 architecture that represents the family of CNN models with no residual blocks, such as AlexNet [26] and VGGNet [47]. We have also implemented ResNet-10 and MobileNetV2 based on their respective standard architectures, representing models with residual block of different complexity and layouts. Our RNN architecture consists of an embedding layer, two LSTM layers, and an output layer with softmax activation.

We have trained these model architecture on various datasets of different complexities, such as MNIST [4], a dataset of hand-written digits with classes 0-9, Fashion MNIST [56] (FMNIST), a dataset with ten fashion classes, the digit section of Kuzushiji MNIST [1] (KMNIST), a dataset of hand-written Japanese digits 0-9, and SVHN [40], a dataset of real-world images for 10-class classification of digits. We trained ResNet-10 and MobileNetV2 model architectures on more complex datasets such as CIFAR-10 [25], consisting of 32×32 color images in 10 different classes, and Caltech-101 [27], consisting 224×224 color images belonging to 102 different object categories. Lastly, we used Reuters [30] and IMDB [38] datasets for training the RNN models. Reuters is a dataset for 90-class classifiers for documents with news articles, and IMDB is a dataset for binary sentiment classification for movie reviews.

Table 2. Mure *vs.* state-of-the-art approaches in terms of speed gain over vanilla approach. Mutation score loss is reported for lossy approaches.

Architecture	Dataset	Mure	DM#		BSS	
		Gain	Gain	Loss	Gain	Loss
LeNet-5	SVHN	58.96%	60.90%	2.75%	80.86%	1.77%
MobileNetV2	Caltech-101	33.28%	50.65%	3.73%	97.27%	10.02%
	CIFAR-10	65.36%	85.39%	2.06%	96.44%	3.58%
ResNet10	Caltech-101	44.42%	79.05%	6.07%	94.41%	4.30%
	CIFAR-10	75.57%	88.91%	4.04%	95.20%	3.69%
RNN	IMDB	17.90%	56.56%	2.31%	94.14%	0.12%
	Reuters	16.29%	23.67%	2.09%	64.46%	11.46%

As indicated by the column "Scope" in the table, we are using more complex models for comparing Mure to state-of-the-art approaches, while simpler FCNN and LeNet-5 models are used in RQ2, making it feasible to run Mure and vanilla mutation analysis thousands of times

In the rest of the paper, we use the combination of model name and training dataset identifier to uniquely identify each of the 12 models, *e.g.*, FCNN-FMNIST, denotes the model with FCNN architecture trained on FMNIST dataset.

We have used two identical Dell Precision workstations with AMD Ryzen Threadripper @ 2.7 GHz CPU, 1 TB of RAM to conduct our experiments. Both machines run Ubuntu 22.04.4 LTS and no GPUs are used in our experiments.

5.4 Answering RQ1

In this RQ, we measure the speed-up offered by Mure and compare it to that of two other approaches, namely DM# and BSS. We run these techniques on 7 DNN models of varying sizes and complexities. The results are reported in Table 2. Since DM# and BSS are lossy approaches, in the sense that they accelerate mutation analysis at the cost of some error in the calculated mutation score, for these approaches we have also reported the mutation score error in the table. Since training, as well as mutation generation, involve randomness, we have repeated our measurement 300 times (=10 rounds of training $\times$ 10 rounds of mutation generation $\times$ 3 rounds of mutation testing) and the averaged values are reported in Table 2.

Across the studied architectures, Table 2 shows that while DM# and BSS achieve higher gain than Mure on average (63.59% and 88.97%, respectively, compared to 44.54% of Mure), they do so by incurring some inaccuracy in mutation score. Specifically, we can observe that in this dataset, DM# results in an average of 3.29% mutation score error and BSS incurs a bit more loss of 4.99%, on average. Although the average acceleration by Mure is relatively lower than the other approaches, it does not incur any mutation score error by construction.

These contrasting characteristics suggest different use cases. Lossy techniques such as DM# and BSS are appropriate when mutation testing is used as a fast heuristic signal, *e.g.*, in exploratory or highly iterative workflows where approximate adequacy estimates are sufficient. Mure is more appropriate when preserving the exact mutation testing outcomes is important, *e.g.*, in mutation-guided test prioritization, debugging, or repair. Although the average mutation-score losses of DM# and BSS are moderate in our experiments, they are not uniformly negligible: the maximum observed losses reach 12.88% and 19.13%, respectively. Such deviations may affect downstream analyses and applications. Mure avoids this trade-off by preserving the vanilla mutation testing result exactly while still cutting mutation testing time nearly in half.

To assess the stability of these results, we additionally computed per-model 95% confidence intervals and non-parametric statistical tests across repeated runs. The confidence intervals are narrow across models, which indicates that the observed speedups are stable under stochastic training and mutation generation. Pairwise Mann-Whitney U-tests [39] with Holm correction [16]

show that the differences among approaches are statistically significant for all models. The full confidence intervals table is included in our replication package [8].

5.5 Answering RQ2

To answer RQ2, we measure the speed-up offered by Mure for each round of mutation generation with a certain *mutation generation selection ratio*. As we discussed in §3.1, the amount of mutation in DeepMutation is controlled by its mutation generation selection ratio parameter. The default value for this parameter is 1%, but here we measure Mure speed-up against 3% and 5% mutation generation selection ratio. We observed that values greater than 5% generated so many low-quality mutants that were filtered out during DeepMutation's mutant quality assessment, thereby causing the program to loop for days without generating any mutants.

To account for the inherent randomness in both model training and mutation generation, we trained each model independently 10 times. For each trained model, we generated mutants 10 times for each 1%, 3%, and 5% mutation generation selection ratio values. Finally, Mure is executed 3 times for each mutant set. This resulted in 900 independent measurements per model configuration, substantially reducing the risk that our findings are artifacts of stochastic variation in training convergence, random weight perturbations, *etc*. These data are plotted in Fig. 2, wherein the speed-up values are averaged for all 900 runs.

We can see from the figure that there is an inverse relationship between the mutation generation selection ratio and the acceleration achieved by Mure. When the percentage of mutated neurons is small (*e.g.*, 1%), Mure consistently yields substantial gains, reducing testing cost by approximately 40%–55% for FCNN models and 60%–65% for LeNet-5 models on average. As the percentage increases, these gains diminish monotonically. This trend is expected: mutation generation selection ratio reduces the amount of shared computation between mutants and the original model, thereby shrinking the reuse opportunity that Mure exploits. In other words, as more neurons are mutated, mutations are more likely to affect earlier layers. This shortens the network prefix that can be memoized and, consequently, limits the achievable speed-up.

We also examined dispersion across runs for each mutation generation selection ratio. The boxplots and means-and-95%-CI plots in the replication package [8] show the same monotonic trend as Fig. 2: speedup decreases as the mutation generation selection ratio increases, but the trend is stable rather than being driven by outliers. Additionally, mutation score error remains zero across *all* ratios and models. We would like to emphasize that running Mure on GPU may make the mutation score calculated by Mure to drift from vanilla due to floating-point errors in most GPUs.

These results provide empirical evidence that mutation generation selection ratio does not limit the practicality of Mure. Instead, its impact is gradual and predictable: while increasing mutation ratio reduces memoization potential, Mure continues to offer considerable acceleration across all studied models for any practically useful mutation generation selection ratio.

6 Discussion

6.1 Practical Implication

We would like to emphasize that Mure, by itself, does not define a new testing adequacy criteria nor is it expected to expose failures that vanilla mutation testing would miss. Rather, Mure preserves the exact semantics of vanilla mutation testing while reducing its cost. This distinction matters because DNN mutation analysis is often used not for quantizing test adequacy, but as a source of information for downstream tasks such as test prioritization, adversarial sample generation, robustness evaluation, fault localization and repair, modularity analysis, and data augmentation. In such settings, replacing vanilla mutation testing with a lossy approximation may change the

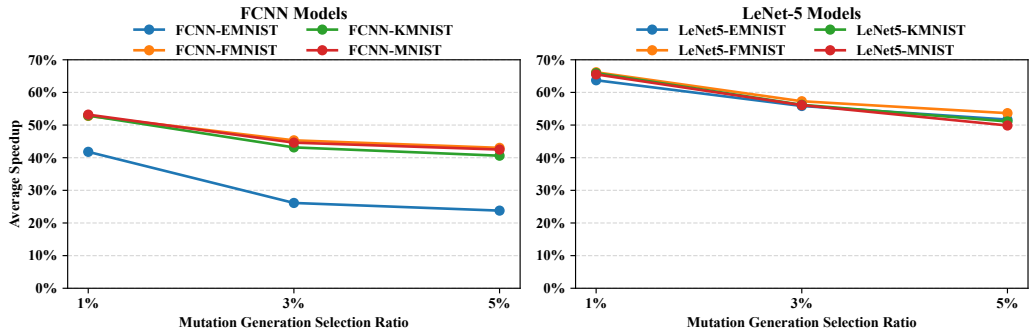


Fig. 2. Average speed-up offered by Mure *vs.* mutation generation selection ratio

downstream behavior of the technique. Mure, instead, allows such workflows to use the *same* mutation-testing outcome as vanilla mutation testing, but *faster*.

6.2 Threats to Validity

While we have provided formal proof for soundness and speed-up for Mure, most factors, such as randomness in higher order mutation and the overhead of mutant chopping and related bookkeeping tasks are difficult to model mathematically. Therefore, we resort to empirical evaluation to take such factors into account, and like most empirical evaluations, our evaluation is subject to certain threats to validity. Additionally, since the proofs are not machine-checked, they may contain error. To mitigate the risks regarding potential errors in our proofs, we have checked our formalization and proofs multiple times, and have included them as part of the main text of the paper.

The first threat to validity of our empirical evaluation comes from the fact that DNN training and mutation generation are stochastic processes, *i.e.*, mutation score calculated for one round of training and mutation generation can be different from that of another round. To account for such randomness, we have trained each model 10 times and each model undergoes 10 rounds of mutation generation, resulting 100 sets of mutants for each model architecture-dataset pair. Additionally, to account for randomness associated with system load, which we expect to be more predictable than randomness in training and/or mutation generation, we have executed Mure and other baseline approaches 3 times. Finally, we have included all the models, measurements, and tools in our replication package [8] so that other researchers can reproduce our results.

Another threat is concerned with the range of mutation generation selection ratios we have studied in RQ2. We observed that increasing mutation generation selection ratio beyond 5% disturbs models so deeply that it turns them to trivial mutants, and we were not able to produce mutants even after running DeepMutation's mutation generation engine for days. It could be possible that further experimentation above the 5% ratio yields models that, given enough time, can generate non-trivial mutants to test Mure's behavior. It may also be fruitful to rethink the mutation strategy in DeepMutation to improve success at higher selection ratios.

7 Related Work

Mutation analysis of DNNs is quite costly, *e.g.*, mutating even the simplest DNN for classification of MNIST images results in hundreds of mutants, each of which must be tested against thousands of test

data points. While some speedup can be obtained using GPU support for numerical calculation, as provided by libraries such as Keras and NumPy, mutation remains an expensive process [7, 10, 19, 22]. Motivated by the many potential practical applications of DNN mutation analysis, researchers have proposed several methods for reducing its costs. Some of these methods are almost directly ported from mutation analysis of conventional programs into DNNs, while others take advantage of the unique characteristics of DNNs. For example, Feng *et al.* [6] and Wang *et al.* [53] identify sufficient subsets of existing mutators from the literature [37, 46] to avoid redundant mutants. Ghanbari *et al.* [10] adopts random mutant selection, and Li *et al.* [31, 33] reduces the cost of mutation testing with higher-order mutants. Meanwhile, the technique introduced by Shen *et al.* [45] relies on the assumption that mutants of a DNN model are more likely to produce different results for the test data points around the decision boundary of the model, so it samples the test data that lie at the decision boundary of the model under test. Ghanbari [7] proposed to accelerate mutation analysis by generating fewer mutants through clustering the neurons. This approach together with mutant clustering based on the similarity of mutated weights have also been studied recently [35]. More recently, Ghanbari and Tavakkol [9] proposed the use of Fourier analysis to generate comparable signatures of mutant behavior to use for mutant clustering and selecting one representative of each mutant for testing.

8 Conclusions and Future Work

This paper introduces Mure, the first provably lossless memoization-based acceleration framework for mutation testing of DNNs. We formalized the execution semantics of memoized mutation testing and established theoretical guarantees that Mure preserves the exact mutation score of vanilla mutation testing, regardless of network architecture, dataset, or mutation operator. We also provide a formal cost analysis and prove that Mure guarantees acceleration under certain conditions on mutant overlap. Our empirical evaluation across a diverse set of DNN architectures show that Mure achieves substantial performance gains by reducing mutation testing cost by 44.54%, on average, without incurring any error on mutation score. Moreover, our analysis of mutation generation selection ratio with different values provide empirical evidence that while increasing mutation generation selection ratio gradually reduces memoization potential, Mure continues to deliver meaningful and predictable acceleration.

This paper opens several promising avenues for future exploration beyond the opportunities pointed out in the paper. We plan to extend Mure to additional classes of architectures, including large transformer-based systems, to investigate whether new forms of structural sharing may further amplify memoization benefits. We also we aim to explore adaptive memoization strategies, dynamic analysis techniques to identify additional reuse opportunities, and hybrid integration with lossy acceleration approaches to provide tunable points in the speed–accuracy design space.

Data Availability

All our measurements and artifacts (such as software, extra tables, and figures) are available in our replication package [8].

Acknowledgments

We would like to thank Anonymous ISSTA 2026 Reviewers for their valuable feedback. The first author is partially supported by the NSF grant #2446393. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF or their employers.

References

[1] Tarin Clanuwat, Mikel Bober-Irizar, Asanobu Kitamoto, Alex Lamb, Kazuaki Yamamoto, and David Ha. 2018. Deep Learning for Classical Japanese Literature. *CoRR* abs/1812.01718 (2018), 1–8. arXiv:1812.01718 http://arxiv.org/abs/1812.01718

[2] George E. Dahl, Jack W. Stokes, Li Deng, and Dong Yu. 2013. Large-scale malware classification using random projections and neural networks. In *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2013, Vancouver, BC, Canada, May 26-31, 2013*. IEEE, 3422–3426. doi:10.1109/ICASSP.2013.6638293

[3] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. 1978. Hints on Test Data Selection: Help for the Practicing Programmer. *Computer* 11, 4 (1978), 34–41. doi:10.1109/C-M.1978.218136

[4] Li Deng. 2012. The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web]. *IEEE Signal Processing Magazine* 29, 6 (2012), 141–142. doi:10.1109/MSP.2012.2211477

[5] Andre Esteva, Alexandre Robicquet, Bharath Ramsundar, Volodymyr Kuleshov, Mark DePristo, Katherine Chou, Claire Cui, Greg Corrado, Sebastian Thrun, and Jeff Dean. 2019. A guide to deep learning in healthcare. *Nature Medicine* 25, 1 (2019), 24–29. doi:10.1038/s41591-018-0316-z

[6] Li-Chao Feng, Xing-Ya Wang, Shi-Yu Zhang, Rui-Zhi Gao, and Zhi-Hong Zhao. 2022. Mutation Operator Reduction for Cost-effective Deep Learning Software Testing via Decision Boundary Change Measurement. *Journal of Internet Technology* 23, 3 (2022), 601–610. https://jit.ndhu.edu.tw/article/view/2705

[7] Ali Ghanbari. 2024. Decomposition of Deep Neural Networks into Modules via Mutation Analysis. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, New York, NY, USA, 1669–1681. doi:10.1145/3650212.3680390

[8] Ali Ghanbari, Ben Greenman, Sasan Tavakkol, and Shibbir Ahmed. 2026. *Provably Lossless Acceleration of DNN Mutation Testing via Memoization (Replication Package)*. doi:10.5281/zenodo.21447443

[9] Ali Ghanbari and Sasan Tavakkol. 2025. Using Fourier Analysis and Mutant Clustering to Accelerate DNN Mutation Testing. In *40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025, Seoul, Korea, Republic of, November 16-20, 2025*. IEEE, 2109–2121. doi:10.1109/ASE63991.2025.00175

[10] Ali Ghanbari, Deepak-George Thomas, Muhammad Arbab Arshad, and Hridesh Rajan. 2023. Mutation-based Fault Localization of Deep Neural Networks. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 1301–1313. doi:10.1109/ASE56229.2023.00171

[11] Yoav Goldberg. 2017. *Neural Network Methods for Natural Language Processing*. Morgan & Claypool Publishers. doi:10.2200/S00762ED1V01Y201703HLT037

[12] Sorin Grigorescu, Bogdan Trasnea, Tiberiu Cocias, and Gigel Macesanu. 2020. A survey of deep learning techniques for autonomous driving. *Journal of Field Robotics* 37, 3 (2020), 362–386. doi:10.1002/rob.21918

[13] Richard G. Hamlet. 1977. Testing Programs with the Aid of a Compiler. *IEEE Trans. Software Eng.* 3, 4 (1977), 279–290. doi:10.1109/TSE.1977.231145

[14] Awni Y. Hannun, Carl Case, Jared Casper, Bryan Catanzaro, Greg Diamos, Erich Elsen, Ryan Prenger, Sanjeev Satheesh, Shubho Sengupta, Adam Coates, and Andrew Y. Ng. 2014. Deep Speech: Scaling up end-to-end speech recognition. *CoRR* abs/1412.5567 (2014), 1–12. arXiv:1412.5567 http://arxiv.org/abs/1412.5567

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 770–778. doi:10.1109/CVPR.2016.90

[16] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. http://www.jstor.org/stable/4615733

[17] Qiang Hu, Yuejun Guo, Maxime Cordy, Mike Papadakis, and Yves Le Traon. 2023. MUTEN: Mutant-Based Ensembles for Boosting Gradient-Based Adversarial Attack. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 1708–1712. doi:10.1109/ASE56229.2023.00042

[18] Qiang Hu, Yuejun Guo, Xiaofei Xie, Maxime Cordy, Mike Papadakis, Lei Ma, and Yves Le Traon. 2023. Aries: Efficient Testing of Deep Neural Networks via Labeling-Free Accuracy Estimation. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 1776–1787. doi:10.1109/ICSE48619.2023.00152

[19] Qiang Hu, Lei Ma, Xiaofei Xie, Bing Yu, Yang Liu, and Jianjun Zhao. 2019. DeepMutation++: A Mutation Testing Framework for Deep Learning Systems. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019, San Diego, CA, USA, November 11-15, 2019*. IEEE, 1158–1161. doi:10.1109/ASE.2019.00126

[20] Xiaowei Huang, Daniel Kroening, Wenjie Ruan, James Sharp, Youcheng Sun, Emese Thamo, Min Wu, and Xinping Yi. 2020. A survey of safety and trustworthiness of deep neural networks: Verification, testing, adversarial attack and defence, and interpretability. *Comput. Sci. Rev.* 37 (2020), 100270. doi:10.1016/J.COSREV.2020.100270

[21] Gunel Jahangirova, Andrea Stocco, and Paolo Tonella. 2021. Quality Metrics and Oracles for Autonomous Vehicles Testing. In *14th IEEE Conference on Software Testing, Verification and Validation, ICST 2021, Porto de Galinhas, Brazil,*

April 12-16, 2021. IEEE, 194–204. doi:10.1109/ICST49551.2021.00030

[22] Gunel Jahangirova and Paolo Tonella. 2020. An Empirical Evaluation of Mutation Operators for Deep Learning Systems. In *13th IEEE International Conference on Software Testing, Validation and Verification, ICST 2020, Porto, Portugal, October 24-28, 2020*. IEEE, 74–84. doi:10.1109/ICST46399.2020.00018

[23] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Trans. Software Eng.* 37, 5 (2011), 649–678. doi:10.1109/TSE.2010.62

[24] Jinhan Kim, Robert Feldt, and Shin Yoo. 2019. Guiding deep learning system testing using surprise adequacy. In *Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25-31, 2019*, Joanne M. Atlee, Tevfik Bultan, and Jon Whittle (Eds.). IEEE / ACM, 1039–1049. doi:10.1109/ICSE.2019.00108

[25] Alex Krizhevsky and Geoffrey Hinton. 2009. *Learning multiple layers of features from tiny images*. Technical Report 0. University of Toronto, Toronto, Ontario. https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

[26] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks. In *Annual Conference on Neural Information Processing Systems*, Peter L. Bartlett, Fernando C. N. Pereira, Christopher J. C. Burges, Léon Bottou, and Kilian Q. Weinberger (Eds.). 1106–1114. doi:10.1145/3065386

[27] Svetlana Lazebnik, Cordelia Schmid, and Jean Ponce. 2006. Beyond Bags of Features: Spatial Pyramid Matching for Recognizing Natural Scene Categories. In *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2006), 17-22 June 2006, New York, NY, USA*. IEEE Computer Society, 2169–2178. doi:10.1109/CVPR.2006.68

[28] Yann LeCun, Yoshua Bengio, and Geoffrey E. Hinton. 2015. Deep learning. *Nature* 521, 7553 (2015), 436–444. doi:10.1038/NATURE14539

[29] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324. doi:10.1109/5.726791

[30] David Lewis. 1987. Reuters-21578 Text Categorization Collection. UCI Machine Learning Repository. doi:10.24432/C52G6M

[31] Yanhui Li, Weijun Shen, Tengchao Wu, Lin Chen, Di Wu, Yuming Zhou, and Baowen Xu. 2022. How higher order mutant testing performs for deep learning models: A fine-grained evaluation of test effectiveness and efficiency improved from second-order mutant-classification tuples. *Inf. Softw. Technol.* 150 (2022), 106954. doi:10.1016/J.INFSOF.2022.106954

[32] Changliu Liu, Tomer Arnon, Christopher Lazarus, Christopher A. Strong, Clark W. Barrett, and Mykel J. Kochenderfer. 2021. Algorithms for Verifying Deep Neural Networks. *Found. Trends Optim.* 4, 3-4 (2021), 244–404. doi:10.1561/2400000035

[33] Jing Liu and Li Song. 2021. Second-Order Mutation Testing Cost Reduction Based on Mutant Clustering using SOM Neural Network Model. In *IEEE 45th Annual Computers, Software, and Applications Conference, COMPSAC 2021, Madrid, Spain, July 12-16, 2021*. IEEE, 974–979. doi:10.1109/COMPSAC51774.2021.00131

[34] Yuteng Lu, Weidi Sun, and Meng Sun. 2022. Towards mutation testing of Reinforcement Learning systems. *J. Syst. Archit.* 131 (2022), 102701. doi:10.1016/J.SYSARC.2022.102701

[35] Lauren Lyons and Ali Ghanbari. 2025. On Accelerating Deep Neural Network Mutation Analysis by Neuron and Mutant Clustering. In *IEEE Conference on Software Testing, Verification and Validation, ICST 2025, Napoli, Italy, March 31 - April 4, 2025*. IEEE, 267–278. doi:10.1109/ICST62969.2025.10989014

[36] Lei Ma, Felix Juefei-Xu, Fuyuan Zhang, Jiyuan Sun, Minhui Xue, Bo Li, Chunyang Chen, Ting Su, Li Li, Yang Liu, Jianjun Zhao, and Yadong Wang. 2018. DeepGauge: multi-granularity testing criteria for deep learning systems. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE 2018, Montpellier, France, September 3-7, 2018*, Marianne Huchard, Christian Kästner, and Gordon Fraser (Eds.). ACM, New York, NY, USA, 120–131. doi:10.1145/3238147.3238202

[37] Lei Ma, Fuyuan Zhang, Jiyuan Sun, Minhui Xue, Bo Li, Felix Juefei-Xu, Chao Xie, Li Li, Yang Liu, Jianjun Zhao, and Yadong Wang. 2018. DeepMutation: Mutation Testing of Deep Learning Systems. In *29th IEEE International Symposium on Software Reliability Engineering, ISSRE 2018, Memphis, TN, USA, October 15-18, 2018*, Sudipto Ghosh, Roberto Natella, Bojan Cukic, Robin S. Poston, and Nuno Laranjeiro (Eds.). IEEE Computer Society, 100–111. doi:10.1109/ISSRE.2018.00021

[38] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. 2011. Learning Word Vectors for Sentiment Analysis. In *Association for Computational Linguistics: Human Language Technologies*, Dekang Lin, Yuji Matsumoto, and Rada Mihalcea (Eds.). The Association for Computer Linguistics, 142–150. https://aclanthology.org/P11-1015/

[39] H. B. Mann and D. R. Whitney. 1947. On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics* 18, 1 (1947), 50–60. http://www.jstor.org/stable/2236101

[40] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y. Ng. 2011. Reading Digits in Natural Images with Unsupervised Feature Learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning 2011*. 4. http://ufldl.stanford.edu/housenumbers/nips2011_housenumbers.pdf

[41] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. 2019. Chapter Six - Mutation Testing Advances: An Analysis and Survey. *Adv. Comput.* 112 (2019), 275–378. doi:10.1016/BS.ADCOM.2018.03.015

[42] Kexin Pei, Yinzhi Cao, Junfeng Yang, and Suman Jana. 2017. DeepXplore: Automated Whitebox Testing of Deep Learning Systems. In *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28-31, 2017*. ACM, 1–18. doi:10.1145/3132747.3132785

[43] Alessandro Viola Pizzoleto, Fabiano Cutigi Ferrari, Jeff Offutt, Leonardo Fernandes, and Márcio Ribeiro. 2019. A systematic literature review of techniques and metrics to reduce the cost of mutation testing. *J. Syst. Softw.* 157 (2019), 110388. doi:10.1016/J.JSS.2019.07.100

[44] Mark Sandler, Andrew G. Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 4510–4520. doi:10.1109/CVPR.2018.00474

[45] Weijun Shen, Yanhui Li, Yuanlei Han, Lin Chen, Di Wu, Yuming Zhou, and Baowen Xu. 2021. Boundary sampling to boost mutation testing for deep learning models. *Inf. Softw. Technol.* 130 (2021), 106413. doi:10.1016/J.INFSOF.2020.106413

[46] Weijun Shen, Jun Wan, and Zhenyu Chen. 2018. MuNN: Mutation Analysis of Neural Networks. In *2018 IEEE International Conference on Software Quality, Reliability and Security Companion, QRS Companion 2018, Lisbon, Portugal, July 16-20, 2018*. IEEE, 108–115. doi:10.1109/QRS-C.2018.00032

[47] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). 1–14. http://arxiv.org/abs/1409.1556

[48] Jeongju Sohn, Sungmin Kang, and Shin Yoo. 2023. Arachne: Search-Based Repair of Deep Neural Networks. *ACM Trans. Softw. Eng. Methodol.* 32, 4 (2023), 85:1–85:26. doi:10.1145/3563210

[49] Florian Tambon, Foutse Khomh, and Giuliano Antoniol. 2023. A probabilistic framework for mutation testing in deep neural networks. *Inf. Softw. Technol.* 155 (2023), 107129. doi:10.1016/J.INFSOF.2022.107129

[50] Macario Polo Usaola and Pedro Reales Mateo. 2010. Mutation Testing Cost Reduction Techniques: A Survey. *IEEE Softw.* 27, 3 (2010), 80–86. doi:10.1109/MS.2010.79

[51] Bo Wang, Yingfei Xiong, Yangqingwei Shi, Lu Zhang, and Dan Hao. 2017. Faster mutation analysis via equivalence modulo states. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis, Santa Barbara, CA, USA, July 10 - 14, 2017*, Tevfik Bultan and Koushik Sen (Eds.). ACM, 295–306. doi:10.1145/3092703.3092714

[52] Jingyi Wang, Guoliang Dong, Jun Sun, Xinyu Wang, and Peixin Zhang. 2019. Adversarial sample detection for deep neural network through model mutation testing. In *Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25-31, 2019*, Joanne M. Atlee, Tevfik Bultan, and Jon Whittle (Eds.). IEEE / ACM, 1245–1256. doi:10.1109/ICSE.2019.00126

[53] Yichun Wang, Zhiyi Zhang, Yongming Yao, and Zhiqiu Huang. 2023. A Fine-Grained Evaluation of Mutation Operators for Deep Learning Systems: A Selective Mutation Approach. In *Proceedings of the 14th Asia-Pacific Symposium on Internetware, Internetware 2023, Hangzhou, China, August 4-6, 2023*, Hong Mei, Jian Lv, Zhi Jin, Xuandong Li, Xiaohu Yang, and Xin Xia (Eds.). ACM, 123–133. doi:10.1145/3609437.3609453

[54] Zan Wang, Hanmo You, Junjie Chen, Yingyi Zhang, Xuyuan Dong, and Wenbin Zhang. 2021. Prioritizing Test Inputs for Deep Neural Networks via Mutation Analysis. In *43rd IEEE/ACM International Conference on Software Engineering, ICSE 2021, Madrid, Spain, 22-30 May 2021*. IEEE, 397–409. doi:10.1109/ICSE43902.2021.00046

[55] Huanhuan Wu, Zheng Li, Zhanqi Cui, and Jianbin Liu. 2022. GenMuNN: A mutation-based approach to repair deep neural network models. *Int. J. Model. Simul. Sci. Comput.* 13, 2 (2022), 2341008:1–2341008:17. doi:10.1142/S1793962323410088

[56] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. *CoRR* abs/1708.07747 (2017), 1–6. arXiv:1708.07747 http://arxiv.org/abs/1708.07747

[57] Jie M. Zhang, Mark Harman, Lei Ma, and Yang Liu. 2022. Machine Learning Testing: Survey, Landscapes and Horizons. *IEEE Trans. Software Eng.* 48, 2 (2022), 1–36. doi:10.1109/TSE.2019.2962027

Received 2026-01-29; accepted 2026-04-16